Create empty controller

Steps to configure empty controller with placeholder swagger

Step 1: Creating Swagger for the new endpoint

1. Run the below command to update the **`swagger/mv-common`** folder which contains shared, reusable OpenAPI components used across multiple API endpoints.

```
npm run mv-swagger
```

2. Create the relevant files in the **`swagger`** subfolders for the new endpoint. Reference to Experience API Design Pattern

Subfolder	Description
entities	Contains domain entity/model definitions
resources	Contains individual API endpoint definitions
requests	Contains request body schemas for API endpoints
responses	Contains successful response schema definitions for API endpoints

Example: GET /message:

swagger/entities/message.yaml

```
Message:
  type: object
  description: Simple message response
  properties:
    message:
      type: string
      example: Hello world
  required:
    - message
```

swagger/resources/message.yaml

```
Message
        get:
                description: Get hello world message
        operationId: getMessage
                tags:
                        - Message
                parameters:
                        - $ref: '../mv-common/parameters.yaml#/x-authorization'
                        - $ref: '../mv-common/parameters.yaml#/x-brandId'
                        - $ref: '../mv-common/parameters.yaml#/x-channelType'
                        - $ref: '../mv-common/parameters.yaml#/x-deviceExperience'
                responses:
                        '200':
                                $ref: '../responses/message.yaml#/200SuccessfulGetMessage'
```

swagger/responses/message.yaml

```
200SuccessfulGetMessage:
  description: Successful response
  content:
    application/json:
      schema:
        $ref: '../entities/message.yaml#/Message'
```

3. Add new endpoint path and reference to the Paths section of **`openapi_ref.yaml`**.

Example:

```
/message:
   $ref: './swagger/resources/message.yaml#/Message'
```

4. Run below command which generates the new endpoint documentation in **`openapi.yaml`**.

```
npm run contract
```

Note: Ensure to obtain approval for the Swagger before proceeding to next step.

Step 2: Generating files for the EXP API

Note: Skip this step if you are creating only a shell controller that does not require any downstream integrations.

1. Add any dependencies (e.g. downstream APIs) to the relevant section of **`dependencies.yaml`**.

Example:

```
dependencies:
  apis:
    - name: cap-technolsvcopsmgt-identaccsmgt-identregorch-v1
  serviceProxies:
    - name: user-authorisation-policy-decision-engine-api-v1
  topics: []
  widgets:
    - name: galaxymakerchecker-v1
```

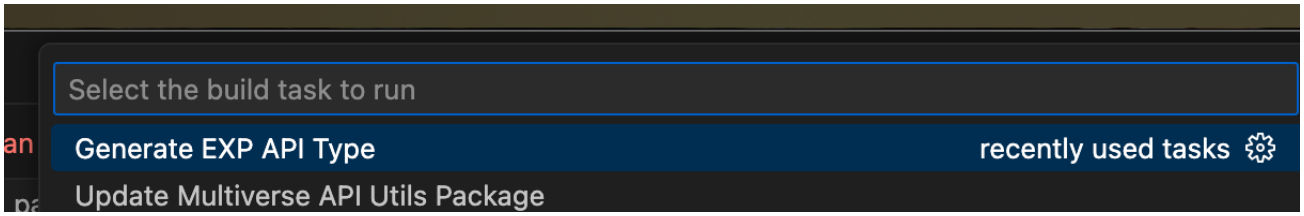
2. Run the below commands to generate the relevant files for the EXP API.

```
npm run policies
npm run routes
npm run api-clients
```

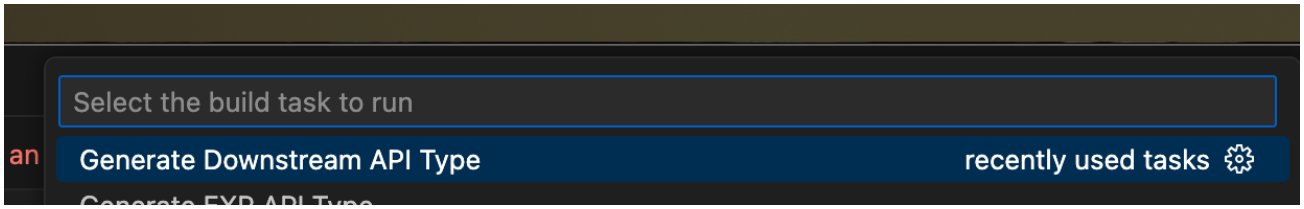
> The **npm run policies** command will update both the **`northEdge.entitlements.json`** and **`southEdge.entitlements.json`** files in the **`api`** folder.
> Please ensure the changes only affect the API you're building.
> And set **"allows": ["*"]** in the **`northEdge.entitlements.json`** for the API you're building.

4. Open the build tasks menu (*Command+Shift+B in VS Code*) and run the following:

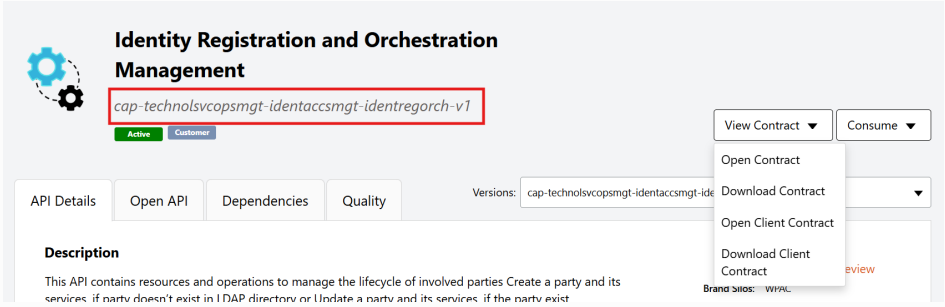
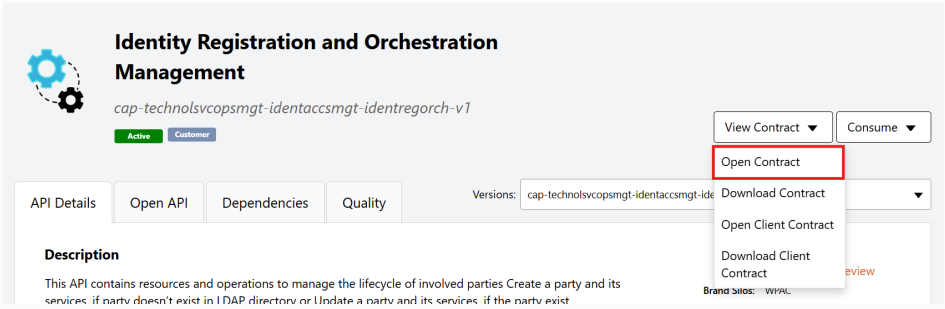
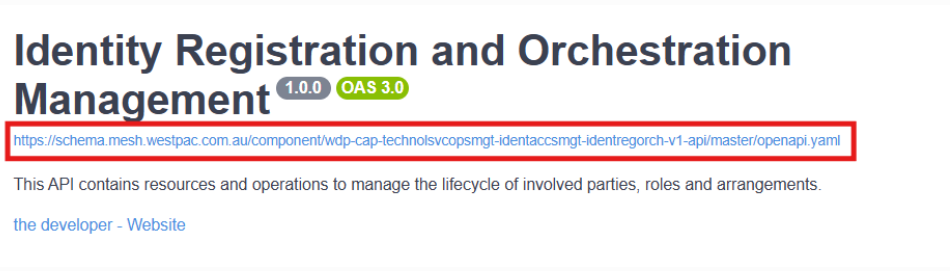
- **`Generate EXP API Type`** to generate the type definitions for the EXP API.



- **`Generate Downstream API Type`** to generate the type definitions for the Downstream API.



The following values will need to be input to generate the downstream API types:

Value	Source	Example	
`downstreamService`	Mesh Connect	*cap-technolsvcopsmgt-identaccsmgt-identregorch-v1*	
`downstreamSwaggerUrl`	Downstream API Contract (Swagger Editor)	https://swagger.mesh.westpac.com.au/?url=https://schema.mesh.westpac.com.au/component/wdp-cap-technolsvcopsmgt-identaccsmgt-identregorch-v1-api/master/openapi.yaml	 

Step 3: Building the Controller

1. Create the **`<EXP_API_Operation_ID>Controller.ts`** file in the **`api/server/api/controllers`** folder.

Template for standard controller shape for EXP API endpoints, key components include:

- Operation logic (call downstream API, transform and normalise response, etc.)
- Exposing custom headers to browser client
- Success path handling + centralised error handling via **`mv-apiutils`**

```
import { NextFunction, Request, Response } from 'express';
import {
handleControllerError,
STATUS_CODES,
accessControlExposeHeaders,
} from '@mesh-tenant-multiverse-ui-common/mv-apiutils';

/**
* <EXP_API_OPERATION_ID>Controller
*
* This is an empty controller used to confirm routing, server configuration,
* and Swagger wiring. It returns a simple "Hello world" payload.
*/
const <EXP_API_OPERATION_ID>Controller = async (
req: Request,
res: Response,
next: NextFunction
) => {
const { logger } = res.locals;

try {
// Expose custom headers
res.setHeader('Access-Control-Expose-Headers', accessControlExposeHeaders);

// TODO: Implement operation logic here (e.g., call downstream API, transform and normalise response data)

// Simple smoketest response
return res
.status(STATUS_CODES.OK)  // use of mv-apiutils STATUS_CODES instead of magic numbers
.json({ message: "Hello world" })
.end();

// Centralised error handling
} catch (error) {
handleControllerError({ error, res, logger });
return next(error);
}
};

module.exports = <EXP_API_OPERATION_ID>Controller;
```

2. Update mock responses (if required). If mock data is needed, update the **`mockoon-data.json`** file in the appropriate environment subfolder under **`api/env`** to define the mock responses for the downstream API(s).

3. Run a simple smoke test by starting the API locally and test using Postman (or curl).

Example: GET /message

```
curl --location 'http://localhost:3007/exp/galaxy/entitlements/v2/message' \
--header 'x-brandId: WBC' \
--header 'x-channelType: Internal' \
--header 'x-deviceExperience: TYPE:app,PLATFORM:windows,VIEWPORT:m,ORIENTATION:landscape' \
--data ''
```